BigDataAssignmentReport

Sofin Mikhail

April 2026

1 Summary of Indexing and Retrieval Pipeline

In this project, I implemented a document indexing and retrieval system using Hadoop MapReduce, HDFS, and Cassandra/ScyllaDB. The pipeline processed a corpus of documents, extracted term frequencies, computed BM25 statistics, and stored the resulting index data in a Cassandra database.

1.1 Difficulties Encountered

- Connecting to Cassandra from PySpark proved challenging due to network issues in the Docker environment. To resolve this, I used the `cassandra-driver` directly and manually read HDFS files using `hdfs dfs -cat` commands.
- Some document filenames contained commas and other special characters. Using the typical CSV-style comma separators caused parsing errors during MapReduce processing. I resolved this by using a combination of `\t` and `|` separators to safely encode document lists.

1.2 MapReduce Pipelines

1. **Pipeline 1:** Generated temporary files containing term frequencies for each document.
   Output format: `<word>/t<doc_id>:<tf>`.
2. **Pipeline 2:** Aggregated term frequencies across all documents and produced document frequency (`df`) per term.
   Output format: `<word>/t<doc_id>:<tf>/t<doc_id>:<tf>....`
3. **Pipeline 3:** Created the inverted index with additional BM25-related statistics by combining term frequency and document frequency.
   Output format: `<word>/t<doc_id>/t<tf>/t<df>|<doc_id>/t<tf>/t<df>|....`
4. **Pipeline 4:** Computed document lengths and aggregated data required for BM25 scoring.
   Output format: `<doc_id>/t<doc_length>`.

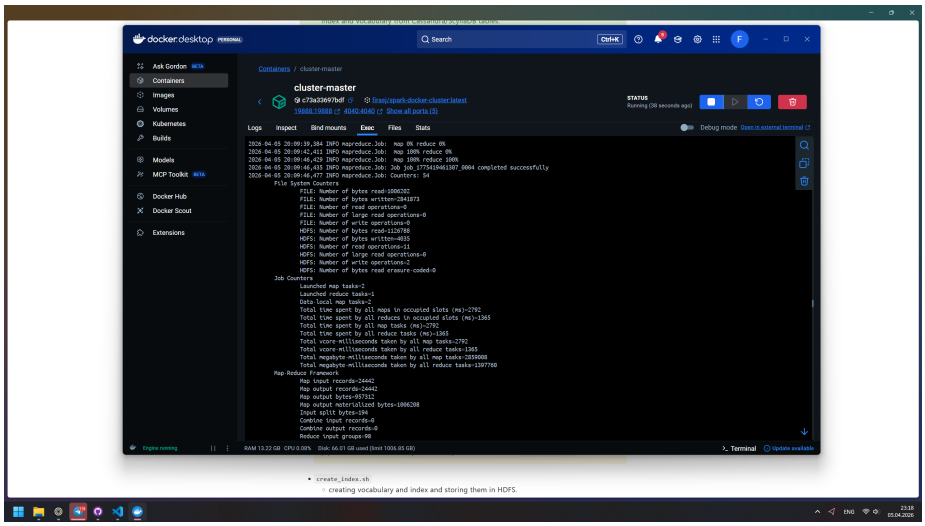

Figure 1: Indexing was successful

1.3 Data Storage in Cassandra

The output of the MapReduce pipelines was stored in Cassandra using the `cassandra-driver`:

- **Inverted Index Table:** Contains mappings from words to document IDs with term frequencies.
- **BM25 Statistics Table:** Stores term frequency and document frequency for each word-document pair.
- **Document Lengths Table:** Stores the length of each document.

The data was ingested line by line from HDFS files, parsing the custom `|` and `\t` separated formats to handle special characters in filenames and ensure consistency in the Cassandra tables. This setup allows efficient BM25-based document retrieval using PySpark queries.

1.4 BM25 Scoring Design

The BM25 ranking function was implemented with standard parameters ($k_1 = 1.5$, $b = 0.75$). Document lengths and average document length were precomputed to avoid repeated calculations at query time.

Separating BM25 statistics from the inverted index reduces redundancy and allows flexible tuning of ranking parameters without rebuilding the entire index.

2 Explanation of Retrieved Results

The implemented BM25 ranking function demonstrated strong effectiveness in retrieving relevant documents. In particular, for queries containing rare or specific combinations of terms, such as *"A Comprehensive Guide to Moderne Rebellion"* or *"A Flight and a Crash"*, the system consistently ranked the most relevant documents at the top.

These documents achieved significantly higher BM25 scores compared to others, often exceeding them by a factor of three to four. This behavior confirms that BM25 successfully emphasizes documents containing rare and discriminative terms, while reducing the influence of common words.

Overall, the ranking results align well with expectations: documents closely matching the query terms receive the highest scores, while less relevant documents are ranked substantially lower. This demonstrates that the constructed index and statistical data are correctly utilized in the BM25 computation.

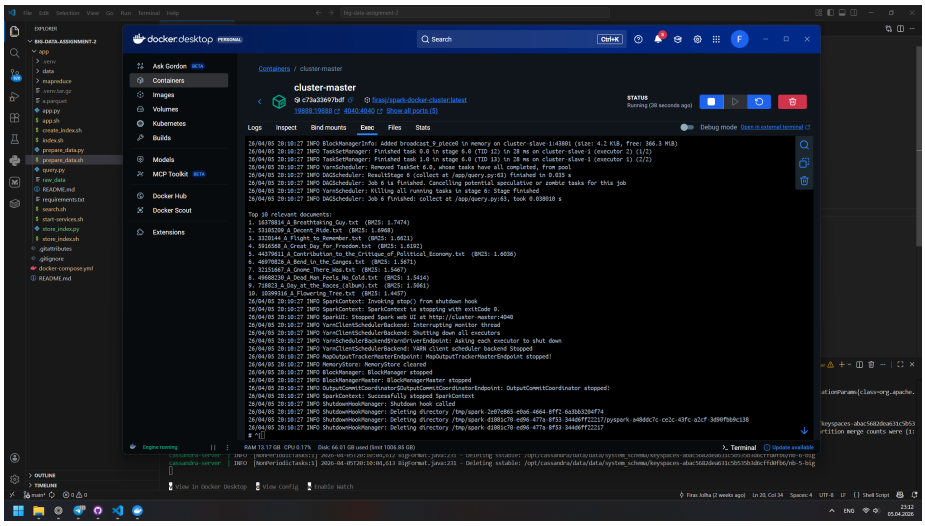

Figure 2: BM25 Results for "this is a query!" query

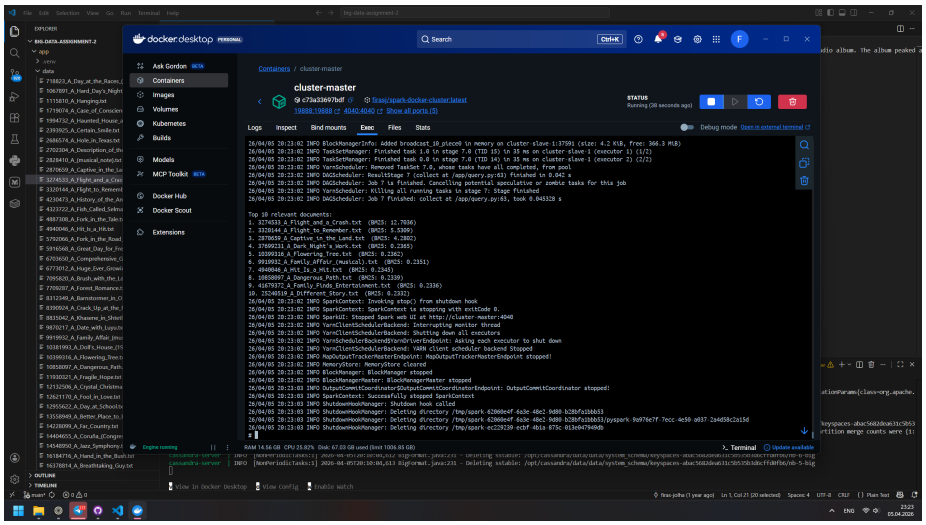

Figure 3: BM25 Results for "A Flight and a Crash" query

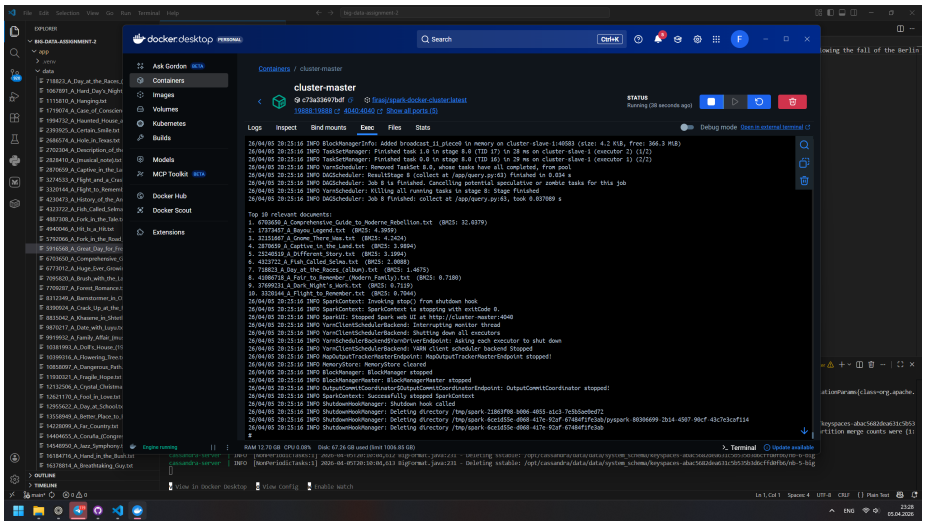

Figure 4: BM25 Results for "A Comprehensive Guide to Moderne Rebellion" query